NetTalk ARMi Fingerprint Login

Ingreso con huella, PIN o bloqueo de pantalla (WebAuthn / claves de paso) para servidores web NetTalk

Manual del programador

Instalación del template, qué graba el instalador y cómo incorporarlo paso a paso a una aplicación NetTalk Server.

Versión del documento: 1.0 (borrador)

Fecha: 1 de octubre de 2026

Probado con: Clarion 11.1 y NetTalk, en PC (Chrome / Edge con Windows Hello) y en Android (Chrome) sobre HTTPS.

Contenido

1. Qué es NetTalk ARMi Login 4
1.1 Qué incluye 4
1.2 Qué no hace 4
1.3 Requisitos 4
2. Resumen del proceso 4
2.1 El código de la huella: cómo viaja el dato 5
3. Qué hace el instalador 6
3.1 Archivos que se copian 6
3.2 Qué hace el template al generar la aplicación 6
3.3 Qué NO toca el instalador 7
3.4 Qué graba el template y la aplicación después 7
4. Incorporar el template a una aplicación NetTalk Server 8
Paso 1. Comprobar que el template está registrado 8
Paso 2. El script del navegador 8
Paso 3. Agregar la extension global 9
Paso 4. Extension del procedimiento de login 9
Paso 5. Código de sesión del usuario (embed) 10
Paso 6. Extension Offer en la página de destino 10
Paso 7. Textos y traducción 11
Paso 8. Generar y compilar 11
Paso 9. Probar 11
Variante: programa Multi-DLL 12
5. Referencia de las extensions 12
5.1 NetTalk ARMi Login - activate (global) 12
5.2 NetTalk ARMi Login - fingerprint button on the login form 13
5.3 NetTalk ARMi Login - offer to activate the fingerprint after login 14
6. Cómo funciona por dentro 15
6.1 Alta de una huella 15
6.2 Ingreso con huella 15
6.3 Qué verifica el servidor 15
6.4 Comunicación navegador - servidor 15
7. Puesta en producción 16
7.1 Servidor detrás de un proxy o reenvío 16
8. Aplicación de ejemplo 16
8.1 Cómo usarla 17
9. Solución de problemas 17
9.1 Códigos de error de la clase 18

10. Preguntas frecuentes..... 18
Apéndice A. Glosario 19

Si el índice aparece vacío, hacer clic derecho sobre él y elegir "Actualizar campo".

1. Qué es NetTalk ARMi Login

NetTalk ARMi Login es un conjunto de templates y una clase de Clarion que agrega a una aplicación **NetTalk Server** el ingreso con la huella digital, el PIN o el bloqueo de pantalla del dispositivo del usuario, usando el estándar **WebAuthn** (las "claves de paso" o "passkeys").

El usuario entra una vez con usuario y contraseña, la aplicación le ofrece activar la huella en ese dispositivo y, desde entonces, puede ingresar tocando un botón y confirmando con su huella o PIN, sin escribir nada.

1.1 Qué incluye

- **Una clase** (`NetWebAuthnClass`) que genera desafíos, verifica firmas ES256 con el CNG de Windows (`bcrypt.dll`) y guarda las credenciales en un archivo TopSpeed.
- **Tres extensions** (en un único template): una global, una para el procedimiento de login y una para la página a la que se llega después de ingresar.
- **Un script de navegador** (`NetTalk_ARMiLogin.js`) que habla con el dispositivo y con el servidor.
- **Un instalador** que deja todo en las carpetas de Clarion (capítulo 3).

1.2 Qué no hace

- No guarda huellas, PINs, claves privadas ni contraseñas. La huella nunca sale del dispositivo; el servidor solo guarda la **clave pública**.
- No reemplaza el login con usuario y contraseña: lo complementa. Ambos siguen funcionando.
- No depende de servicios externos ni de otros lenguajes: todo corre dentro del servidor Clarion.

1.3 Requisitos

Requisito	Detalle
Entorno de desarrollo	Clarion 10 o 11.x o 12 con NetTalk instalado. La aplicación debe ser un **NetTalk Server** con un procedimiento de login hecho con el template NetWebForm.
Servidor de producción	Windows 7 / Server 2008 R2 o posterior (usa `bcrypt.dll`, que es parte de Windows).
HTTPS y dominio	WebAuthn **solo funciona en un origen seguro**: HTTPS con certificado válido, o `http://localhost`. Además la clave se asocia a un **nombre de dominio**: no sirve una dirección IP.
Dispositivos	Navegadores modernos (Chrome, Edge, Safari, Firefox) con huella, PIN o bloqueo de pantalla configurado. En Chrome de Android no funciona en modo incógnito.

2. Resumen del proceso

Incorporar el template a una aplicación lleva nueve pasos, descritos en el capítulo 4:

1. Instalar el template en Clarion (lo hace el instalador, capítulo 3) y comprobar que está registrado.
2. Comprobar que `NetTalk_ARMiLogin.js` se copió a la carpeta `web\scripts` del ejecutable (lo hace el template al generar, ver 3.2).
3. Agregar la extension **global** a la aplicación.

4. Agregar la extension Form al procedimiento **LoginForm** (NetWebForm) y completar sus prompts.
5. Escribir el código de **sesión** en el embed `ARMi Login: fingerprint verified`.
6. Agregar la extension **Offer** a la página de destino posterior al login.
7. Revisar o traducir los textos de las extensions.
8. Generar y compilar la aplicación.
9. Probar el ingreso con huella.

2.1 El código de la huella: cómo viaja el dato

Esto es lo que más conviene entender antes de configurar las extensions. Al activar la huella, el servidor guarda junto con ella un valor que vos elegís: el código de la huella. Cuando después alguien entra con esa huella, el servidor lo devuelve a tu login. Ese valor puede ser cualquier dato que identifique al usuario: su nombre, su e-mail o un ID interno (por ejemplo 10015). No tiene por qué ser lo que el usuario escribe en el login.

Momento	Dónde se configura	Qué pasa con el código
Activación: el usuario ya logueado acepta "Activar huella"	Extension Offer, prompt Value to store in the passkey	La expresión se evalúa (por ejemplo p_web.GetSessionValue('LOGIN_ID')) y el resultado se guarda con la huella en NetTalkPasskeys.tps (campo UserKey).
Ingreso con huella: el usuario toca "Login with fingerprint"	Extension Form, prompt Variable to receive the passkey code	El servidor encuentra la huella, toma el código guardado y lo deja en esa variable (por ejemplo Loc:Login), y también como p_web.GetValue('loc:login') y como valor de sesión loc:login.
Después del ingreso	Embed ARMi Login: fingerprint verified (en el procedimiento de login)	Tu código usa p_web.GetValue('loc:login') para encontrar al usuario y cargar su sesión: el mismo código que el login normal ejecuta al presionar OK, sin la comparación de la contraseña.

Ejemplo: si tu login normal identifica al usuario por un ID interno, guardá ese ID (por ejemplo 10015) al activar la huella y, en el embed, buscá al usuario por ID usando p_web.GetValue('loc:login'). Si lo identifica por e-mail, guardá el e-mail. Lo único que importa es que el dato que se guarda sea el mismo con el que el embed busca.

- El valor debe ser único para cada usuario y no puede estar vacío.
- Conviene que sea estable: si el dato de un usuario cambia (por ejemplo, su e-mail), las huellas ya activadas siguen devolviendo el valor anterior; hay que borrarlas y activarlas de nuevo (ver la pregunta frecuente sobre este caso).
- En el embed el valor se lee con p_web.GetValue('loc:login') (el nombre de la variable elegida, en minúsculas). La variable local del formulario solo vive mientras se procesa el OK del login.

> **Importante:** Los dos prompts hablan del mismo dato: lo que ponés en Value to store in the passkey (extension Offer) es lo que recibís en Variable to receive the passkey code (extension Form). Si ese dato no es el que tu embed espera para buscar al usuario, el ingreso con huella no va a encontrarlo.

3. Qué hace el instalador

El instalador copia los archivos del template a las carpetas de Clarion y deja disponible el template en el IDE. **No modifica ninguna aplicación (.app), diccionario ni otros templates.** Esta sección detalla exactamente qué graba y dónde.

3.1 Archivos que se copian

Archivo	Carpeta de destino	Para qué sirve
`NetTalk_ARMiLogin.inc`	`<Clarion>\Accessory\LibSrc\Win\`	Declaración de la clase `NetWebAuthnClass`.
`NetTalk_ARMiLogin.clw`	`<Clarion>\Accessory\LibSrc\Win\`	Implementación de la clase (desafíos, verificación de firma con CNG, almacén de credenciales).
`NetTalk_ARMiLogin.tpl`	`<Clarion>\Accessory\Template\Win\`	Cabecera del template; es el archivo que se registra en el IDE.
`NetTalk_ARMiLogin.tpw`	`<Clarion>\Accessory\Template\Win\`	Las tres extensions y sus prompts.
`bcrypt.lib`	`<Clarion>\Accessory\Lib\`	Librería de importación de `bcrypt.dll`. Sin ella el programa no enlaza.
`NetTalk_ARMiLogin.js`	`<Clarion>\Accessory\LibSrc\Win\`	Script del navegador (copia maestra). **No se instala dentro de ninguna aplicación**: el template lo copia solo a `web\scripts` del ejecutable de cada app al generarla (ver 3.2).
Este manual (.docx / .pdf)	`<Clarion>\ARMi\Nettalk_ARMiLogin`	Documentación.
Aplicación de ejemplo	`<Clarion>\ARMi\Nettalk_ARMiLogin`	Ver el capítulo 8.

Las carpetas `Accessory\LibSrc\Win`, `Accessory\Template\Win` y `Accessory\Lib` forman parte de las rutas de búsqueda estándar de Clarion (archivo de redirección `.red`), por eso el compilador, el enlazador y el IDE encuentran los archivos sin configurar nada. Verificado: el enlazador encuentra `bcrypt.lib` en `Accessory\Lib`. En cambio, una copia en la raíz de Clarion (`<Clarion>\bcrypt.lib`) **no** la encuentra.

3.2 Qué hace el template al generar la aplicación

El programador no tiene que copiar nada a mano. Cada vez que se **genera** la aplicación (Generate, o Make cuando el IDE regenera antes de compilar) el template ejecuta estas acciones, de la misma forma que NetTalk copia sus propios archivos web:

1. Comprueba que `bcrypt.lib` exista en `<Clarion>\Accessory\Lib` (o en `<Clarion>\Lib`). Si no está, muestra un error en el IDE: "bcrypt.lib not found ... Reinstall NetTalk ARMi Login".
2. Busca `NetTalk_ARMiLogin.js` en `<Clarion>\Accessory\LibSrc\Win`. Si no está, muestra un error.
3. Copia el script a la carpeta `scripts` dentro de la carpeta web del ejecutable. La carpeta web es la misma que usa NetTalk (prompt **(To) Web Folder** de su extension global, por defecto `web`), ubicada

respecto de la carpeta donde el archivo de redirección de Clarion graba el ejecutable (por ejemplo .\Exe), que es donde el servidor la busca al correr. Si `scripts` no existe, la crea.

4. Si en NetTalk está tildado **Do not copy Web Folder**, no copia nada y avisa en el IDE que el script debe copiarse a mano.

Para copiar, el template crea un archivo temporal `ARMiLoginCopy.Bat` (el redir de Clarion lo ubica, por ejemplo, en .\tmp), lo ejecuta con su ruta completa y lo borra. Al terminar escribe en la ventana de mensajes del IDE: `NetTalk ARMi Login: NetTalk_ARMiLogin.js copied to web\scripts`.

> **Nota:** `bcrypt.lib` no se copia: Clarion ya busca los `.lib` en `Accessory\Lib` y el template agrega `bcrypt.lib` a las bibliotecas del proyecto. Lo único que hace con él es verificar que esté.

> **Importante:** La copia se hace al generar. Si se actualiza el instalador (un `.js` nuevo), hay que regenerar cada aplicación para que reciba la versión nueva.

3.3 Qué NO toca el instalador

- Ninguna aplicación `.app`, `.txa`, proyecto `.cwproj` ni diccionario `.dct`.
- Los templates y clases de NetTalk, StringTheory ni de Clarion.
- La carpeta `web` de ninguna aplicación ni sus datos.

3.4 Qué graba el template y la aplicación después

Una vez que el programador usa el template en una aplicación, ocurre lo siguiente (no lo hace el instalador):

Qué	Dónde	Detalle
Biblioteca en el proyecto	Archivo `.cwproj` de la aplicación	Al generar, el template agrega `bcrypt.lib` a la lista de bibliotecas.
Script en la aplicación	`<carpeta del exe>\web\scripts\NetTalk_ARMiLogin.js`	Copiado por el template al generar (ver 3.2). El archivo temporal `ARMiLoginCopy.Bat` se borra al terminar.
`INCLUDE` de la clase	Código generado (`.clw` principal)	`INCLUDE('NetTalk_ARMiLogin.inc'),ONCE`
Objeto `WAAuth`	Datos locales de cada procedimiento con extension	Una instancia por solicitud: no se comparte entre hilos.
Credenciales	`NetTalkPasskeys.tps` (nombre configurable)	Lo crea el servidor en el primer alta, en la carpeta donde corre el `.exe`.
Marcas en el navegador	`localStorage` del sitio	`netwebauthn_known` (este dispositivo ya activó la huella) y `netwebauthn_declined` (eligió "no preguntar más").
Desafío temporal	Sesión de NetTalk	`_wa_chal_` (de un solo uso) y `_wa_done_` (marca interna de respuesta ya enviada).
Log de la aplicación	`p_web.AddLog`	`fingerprint login <usuario>`, `fingerprint login refused: <motivo>`, `fingerprint activated for <usuario>`.

Contenido de NetTalkPasskeys.tps

Un registro por cada huella activada. **Solo contiene claves públicas**; si alguien se llevara el archivo no podría ingresar con él.

Campo	Tipo	Contenido
`CredId`	STRING(256)	Identificador de la credencial (clave única). El dispositivo lo envía al ingresar.
`UserKey`	STRING(255)	Código guardado con la huella; identifica al usuario (lo que el login recibe de vuelta en su variable).
`PublicKey`	STRING(160)	Clave pública ES256 en base64url, con la que se verifica la firma.
`SignCount`	ULONG	Contador de firmas; detecta credenciales clonadas o repetidas.
`Device`	STRING(60)	Plataforma del dispositivo (informativo).
`Created`, `CreatedTime`	LONG	Fecha y hora del alta.
`LastUsed`	LONG	Fecha del último ingreso.

Índices: `ByCred` (clave primaria, única) y `ByUser` (admite duplicados: un usuario puede tener varias huellas, una por dispositivo).

4. Incorporar el template a una aplicación NetTalk Server

> **Nota:** Las capturas corresponden a Clarion 11.1. En otras versiones del IDE los menús pueden variar un poco. Los nombres exactos de las extensions son los de la tabla del capítulo 5.

Paso 1. Comprobar que el template está registrado

1. Abrir el IDE de Clarion y elegir **Setup > Template Registry**.
2. Buscar **NetTalk ARMi Login** en la lista. Si no aparece, usar **Register...** y elegir `<Clarion>\Accessory\Template\Win\NetTalk_ARMiLogin.tpl`.
3. Cerrar la ventana con **Close**.

Paso 2. El script del navegador

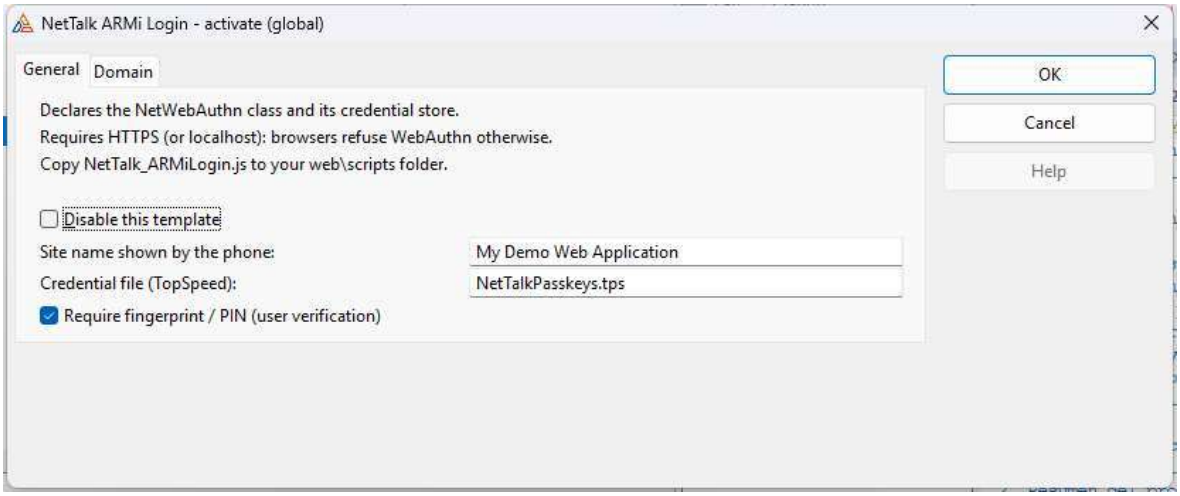
No hay que copiar nada a mano: el template copia `NetTalk_ARMiLogin.js` a la carpeta `web\scripts` del ejecutable (la carpeta donde corre el servidor) cada vez que se genera (ver 3.2). Este paso es solo una **comprobación**, que se hace después de generar por primera vez (paso 8): el archivo debe estar junto a `netweb.js`, en la carpeta que el servidor NetTalk publica como `/scripts/`.

> **Nota:** Para comprobarlo con el servidor en marcha, abrir `http://localhost:<puerto>/scripts/NetTalk_ARMiLogin.js` en el navegador: debe mostrar el código del script y no un error 404.

> **Importante:** Si en NetTalk está tildado **Do not copy Web Folder**, el template no copia el script: copiarlo a mano desde `<Clarion>\Accessory\LibSrc\Win`.

Paso 3. Agregar la extension global

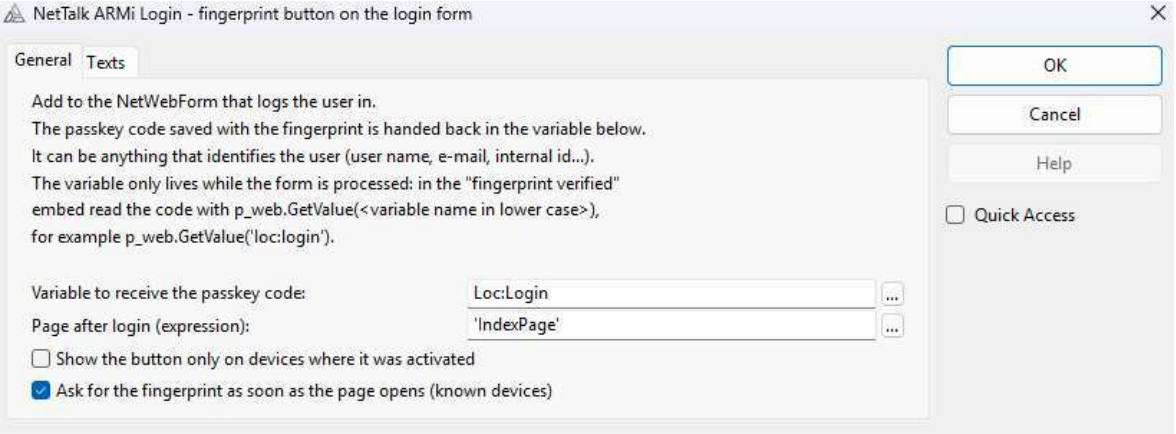
1. Abrir la aplicación en el IDE (árbol de la aplicación).
2. Entrar a **Global Properties** y presionar el botón **Extensions**.
3. Presionar **Insert** y elegir **NetTalk ARMi Login - activate (global)**.
4. Completar la solapa **General**: nombre del sitio (lo que verá el usuario en la ventana de su dispositivo) y archivo de credenciales.
5. Dejar la solapa **Domain** en blanco, salvo que el servidor esté detrás de un proxy (ver el capítulo 7).
6. Aceptar con **OK**.



Paso 4. Extension del procedimiento de login

Este es el procedimiento NetWebForm que muestra usuario y contraseña (en las aplicaciones de ejemplo se llama `LoginForm`).

1. En la lista de procedimientos, abrir el procedimiento de login.
2. Presionar el botón **Extensions** y luego **Insert**.
3. Elegir **NetTalk ARMi Login - fingerprint button on the login form**.
4. En **Variable to receive the passkey code** elegir la variable donde queda el código guardado con la huella (por ejemplo `Loc:Login`). Ese código puede ser cualquier dato que identifique al usuario (nombre, e-mail, un ID interno). La variable solo existe mientras el formulario procesa el OK del login: en el embed del paso 5 el código se lee con p_web.GetValue('loc:login'). El circuito completo está explicado en 2.1.
5. En **Page after login** escribir la página de destino **entre comillas**, por ejemplo `'IndexPage'`. Es una expresión.
6. En la solapa **Texts** revisar o traducir los textos (ver el paso 7).
7. Aceptar con **OK**.



Paso 5. Código de sesión del usuario (embed)

Cuando la huella es válida, el servidor deja el código guardado con la huella en la variable elegida en el paso 4, marca la sesión como iniciada (`p_web.ValidateLogin()`) y ejecuta un **embed** para que el programador cargue los datos de sesión de ese usuario. El código queda disponible como valor web y de sesión, con el nombre de la variable en minúsculas (por ejemplo p_web.GetValue('loc:login')). La variable local no conviene usarla dentro del embed.

1. Abrir el **Embeditor** del procedimiento de login.
2. Buscar el embed **ARMi Login: fingerprint verified**, dentro de **Routines > ARMi Login**.
3. Escribir allí el mismo código que el login normal ejecuta después de validar la contraseña (variables de sesión, nivel de acceso, fecha de último ingreso, etc.).

Ejemplo genérico (cada aplicación carga sus propias variables de sesión):

```
! El código guardado con la huella (el que identifica al usuario) está en
p_web.GetValue('loc:login').
! p_web.ValidateLogin() ya fue llamado por el template.
p_web.SetSessionValue('LOGIN_ID',clip(p_web.GetValue('loc:login')))
p_web.SetSessionValue('LOGIN_name',clip(MiTabla:Nombre))
p_web.SetSessionLevel(0)
p_web.AddLog('login con huella OK: ' & clip(p_web.GetValue('loc:login')))
```

> **Importante:** El código de este embed **no debe comparar contraseñas**: la identidad ya fue verificada por la firma del dispositivo. Si el login normal busca al usuario en una tabla (por ejemplo por e-mail), hacer esa misma búsqueda aquí usando `p_web.GetValue('loc:login')`. Ojo: el código es el dato que se guardó al activar la huella (puede ser un e-mail, un nombre o un ID): la búsqueda debe usar ese mismo dato. No usar la variable local del formulario (Loc:Login): solo existe mientras se procesa el OK del login.

Paso 6. Extension Offer en la página de destino

Es la página a la que va el usuario después de ingresar (por ejemplo `IndexPage`). Allí se muestra, una sola vez por dispositivo, la ventana "¿Activar huella?" y se atiende el alta.

1. Abrir el procedimiento de la página de destino (NetWebPage o NetWebForm).
2. Presionar **Extensions > Insert** y elegir **NetTalk ARMi Login - offer to activate the fingerprint after login**.
3. En **Show the offer when** dejar `p_web.GetSessionLoggedIn() = 1` (solo se ofrece a usuarios ya logueados).
4. En **Value to store in the passkey** escribir la expresión que devuelve el dato que identifica al usuario (el que después va a recibir el login por huella), por ejemplo `p_web.GetSessionValue('LOGIN_ID')`.
5. Aceptar con **OK**.



> **Importante:** **Importante: El código guardado con la huella es lo que luego recibe el login por huella: debe ser el dato con el que el embed busca al usuario.** Si el alta guarda `juan@correo.com` y el embed busca por nombre (`JUAN`), el ingreso con huella no encontrará al usuario. Usar siempre el mismo dato en los dos lugares. Ver 2.1.

Paso 7. Textos y traducción

Todos los textos que ve el usuario están en la solapa **Texts** de cada extension y pasan por `p_web.Translate()`. Hay dos formas de dejarlos en el idioma de la aplicación:

- Escribirlos directamente en el prompt (lo más simple si la aplicación es de un solo idioma).
- Cargar cada texto en inglés con su traducción en la tabla de traducciones de NetTalk (si la aplicación es multi-idioma).

Paso 8. Generar y compilar

1. Generar la aplicación (**Generate** o **Make**).
2. En la ventana de mensajes del IDE debe aparecer `NetTalk ARMi Login: NetTalk_ARMiLogin.js copied to ...` (paso 2). Si aparece un error de `bcrypt.lib` o del `.js`, la instalación está incompleta (ver 3.1).
3. Compilar. Si el enlazador informa `Unresolved External BCrypt...`, falta `bcrypt.lib` en `<Clarion>\Accessory\Lib`.
4. Copiar el ejecutable y la carpeta `web` (con el `.js`) al lugar desde donde se ejecuta.

Paso 9. Probar

Hacer la primera prueba en la misma PC, con `http://localhost:<puerto>` (es un origen seguro).

1. Abrir la página de login. Debe verse el botón **Login with fingerprint** a la derecha de "Ingresar".
2. Tocar el botón: como el dispositivo aún no activó la huella, aparece un aviso "no activado". Cerrarlo con **OK**.
3. Ingresar con usuario y contraseña. En la página de destino aparece la ventana modal **Activate fingerprint login on this device?**. Elegir **Activate** y confirmar con Windows Hello (PIN o huella).
4. Cerrar sesión y tocar **Login with fingerprint**. Windows muestra la clave del sitio; confirmar. Se ingresa directamente a la página de destino.

Para repetir la prueba desde cero: borrar `NetTalkPasskeys.tps` del servidor y ejecutar `localStorage.clear()` en la consola del navegador (F12 > Console).

> **Nota:** Para probar desde un teléfono hace falta HTTPS con un nombre de dominio. Durante el desarrollo se puede usar un túnel (por ejemplo ngrok) hacia el puerto del servidor local, recordando que **expone el servidor a internet mientras esté abierto**.

Variante: programa Multi-DLL

En un programa Multi-DLL la clase NetWebAuthn se compila y se exporta desde **una sola** DLL (la **DLL raíz**); todas las demás DLL y el EXE la importan desde ahí. Se configura en la solapa **Multi-DLL** de la extension global. Un programa formado por un único EXE no necesita nada: dejar la solapa como está.

1. Elegir la DLL raíz: la que va a contener la clase (normalmente la que ya contiene las clases comunes).
2. Agregar la extension **global** a **todas** las aplicaciones del programa (DLL raíz, las otras DLL y el EXE), como en el paso 3, y completar la solapa **General** igual en todas.
3. En cada aplicación abrir la solapa **Multi-DLL** y tildar **This is part of a Multi-DLL program**. En una DLL viene tildado solo; **en el EXE hay que tildarlo a mano**.
4. Solo en la DLL raíz tildar **Export the NetWebAuthn class from this DLL**. En todas las demás aplicaciones dejarlo sin tildar.
5. Tildar **Copy NetTalk_ARMiLogin.js to the web folder when generating** únicamente en la aplicación que ejecuta el servidor web NetTalk (normalmente el EXE). En las DLL viene sin tildar.
6. Agregar las extensions **Form** y **Offer** en los procedimientos que corresponda, en la aplicación que los contenga (pasos 4 a 6).
7. Generar y compilar primero la DLL raíz y después las otras DLL y el EXE.

> **Nota:** `bcrypt.lib` solo la enlaza la DLL raíz (o la única aplicación en un programa común), por eso las demás aplicaciones no la necesitan.

> **Importante:** Tildar **Export the NetWebAuthn class from this DLL** en **una sola** DLL. Si se tilda en dos, el enlazador informa exportaciones duplicadas; si no se tilda en ninguna, el EXE informa errores **Unresolved External** sobre métodos de `NetWebAuthnClass`.

5. Referencia de las extensions

5.1 NetTalk ARMi Login - activate (global)

Se agrega una sola vez por aplicación, en las propiedades globales.

Prompt	Tipo	Valor por defecto	Descripción
Disable this template	Casilla	No	Desactiva toda la generación del template en esta aplicación.
Site name shown by the phone	Texto	`My Web Application`	Nombre del sitio que muestra el dispositivo al crear la clave. Obligatorio.
Credential file (TopSpeed)	Texto	`NetTalkPasskeys.tps`	Archivo de credenciales. Puede incluir una ruta completa. Obligatorio.
Require fingerprint / PIN	Casilla	Sí	Exige que el usuario se verifique en el dispositivo (huella, PIN). Dejarlo marcado.

Prompt	Tipo	Valor por defecto	Descripción
RP id (domain)	Expresión	vacío	Dominio de la clave. Vacío = se toma del Host de cada solicitud, sin el puerto.
Expected origin	Expresión	vacío	Origen esperado. Vacío = `https://` + Host (o `http://` si es localhost).
This is part of a Multi-DLL program	Casilla	Sí en una DLL, No en un EXE	**Solapa Multi-DLL.** Indica al template que el programa es Multi-DLL. Tildarlo en todas las aplicaciones del programa, EXE incluido.
Export the NetWebAuthn class from this DLL	Casilla	No	**Solapa Multi-DLL.** Solo en la DLL raíz: compila la clase y la exporta. Las demás aplicaciones la importan.
Copy NetTalk_ARMiLogin.js to the web folder when generating	Casilla	Sí en un EXE, No en una DLL	**Solapa Multi-DLL.** Dejarlo tildado solo en la aplicación que ejecuta el servidor web NetTalk (ver 3.2).

5.2 NetTalk ARMi Login - fingerprint button on the login form

Se agrega al procedimiento NetWebForm del login. Requiere la extension global.

Prompt	Tipo	Valor por defecto	Descripción
Variable to receive the passkey code	Campo	(obligatorio)	Variable que recibe el código guardado con la huella (p. ej. `Loc:Login`). Solo vive durante el procesamiento del formulario: en el embed leer el código con p_web.GetValue('loc:login').
Page after login	Expresión	`'IndexPage'`	Página a la que se redirige tras el ingreso. Va entre comillas.
Show the button only on devices where it was activated	Casilla	No	Oculta el botón en dispositivos sin la marca de activación.
Ask for the fingerprint as soon as the page opens	Casilla	No	En dispositivos ya activados, abre la verificación al cargar la página.
Button	Texto	`Login with fingerprint`	Texto del botón.

Prompt	Tipo	Valor por defecto	Descripción
Login failed	Texto	`The fingerprint could not be verified. Use your user and password.`	Mensaje cuando el servidor rechaza el ingreso.
Not supported	Texto	`This browser or connection does not support fingerprint login (HTTPS is required).`	Navegador o conexión sin WebAuthn.
Not activated yet	Texto	`Fingerprint login is not activated on this device yet. Log in with your user and password and you will be offered to activate it.`	Aviso al tocar el botón en un dispositivo sin activar.
Not registered on the server	Texto	`This fingerprint is no longer registered on the server. Log in with your user and password and activate it again.`	El dispositivo tiene una clave que el servidor ya no conoce.
Button OK / Button try anyway	Texto	`OK` / `Try anyway`	Botones del aviso "no activado".

Embed que agrega: **ARMi Login: fingerprint verified** (**Routines > ARMi Login**).

5.3 NetTalk ARMi Login - offer to activate the fingerprint after login

Se agrega a la página de destino posterior al login (NetWebPage o NetWebForm). Requiere la extension global.

Prompt	Tipo	Valor por defecto	Descripción
Show the offer when	Expresión	`p_web.GetSessionLoggedIn() = 1`	Condición para mostrar la oferta.
Value to store in the passkey	Expresión	`p_web.GetSessionValue('loc:login')`	Código que se guarda con la huella; identifica al usuario. El login por huella lo devuelve en la variable del paso 4 (p_web.GetValue('loc:login')).
Question / Yes / Later / Never	Texto	`Activate fingerprint login on this device?` / `Activate` / `Not now` / `Do not ask again`	Textos de la ventana modal de oferta.
Done / Failed	Texto	`Fingerprint login activated.` / `The fingerprint could not be activated.`	Resultado del alta.

Prompt	Tipo	Valor por defecto	Descripción
Already activated	Texto	`Fingerprint login is already activated on this device.`	Aviso cuando este dispositivo ya tenía activada la huella (el navegador no permite crear una clave repetida).
Not supported	Texto	`This browser or connection does not support fingerprint login.`	Navegador sin WebAuthn.

6. Cómo funciona por dentro

6.1 Alta de una huella

1. El usuario ingresa con usuario y contraseña y llega a la página de destino.
2. El script ve que el dispositivo no está marcado como activado y muestra la ventana de oferta.
3. Al aceptar, el navegador pide al servidor un desafío (`_wa_=ropts`).
4. El dispositivo crea un par de claves (ES256) y firma con la huella o el PIN. La clave privada queda en el dispositivo.
5. El navegador envía al servidor la clave pública y los datos firmados (`_wa_=rverify`).
6. El servidor verifica el desafío, el origen, el dominio y los indicadores, y guarda la clave pública en `NetTalkPasskeys.tps`.

6.2 Ingreso con huella

1. El usuario toca **Login with fingerprint**. El navegador pide un desafío al servidor (`_wa_=lopts`).
2. El dispositivo muestra las claves del sitio y el usuario confirma con huella o PIN. El dispositivo firma el desafío.
3. El navegador envía la firma (`_wa_=lverify`).
4. El servidor busca la credencial, verifica la firma con la clave pública, comprueba que el contador creció y carga el código guardado en la variable del login.
5. Llama a `p_web.ValidateLogin()`, ejecuta el embed del paso 5 y responde con la página de destino; el navegador redirige.

6.3 Qué verifica el servidor

- El **tipo** de operación (`webauthn.create` / `webauthn.get`) y que el **desafío** coincida con el emitido en esa sesión (de un solo uso).
- Que el **origen** y el hash del **dominio** correspondan a esta aplicación.
- Que el usuario haya estado presente y se haya **verificado** (huella o PIN), si el prompt lo exige.
- La **firma** ES256 con la clave pública guardada.
- Que el **contador de firmas** crezca (los dispositivos que siempre informan 0 se aceptan).

6.4 Comunicación navegador - servidor

El script habla con la misma página que tiene la extension, agregando el parámetro `_wa_`. Las respuestas son JSON pequeño (`{"ok":1,...}`):

Parámetro	Dónde se atiende	Respuesta
`_wa_=lopts`	Procedimiento de login	Desafío y dominio para el ingreso.
`_wa_=lverify`	Procedimiento de login	`{"ok":1,"next":...}` si el ingreso es válido; `err` = `auth` o `unknown` si no.
`_wa_=ropts`	Página de destino	Desafío, datos del usuario y credenciales ya registradas (para no duplicar).
`_wa_=rverify`	Página de destino	`{"ok":1}` si el alta se guardó.

7. Puesta en producción

Antes de publicar, el servidor debe estar accesible por **HTTPS con certificado válido** y un **nombre de dominio**.

1. Compilar la aplicación en la PC de desarrollo (donde está `bcrypt.lib`) y copiar el `.exe` nuevo al servidor.
2. Desplegar la carpeta `web` del ejecutable: ya contiene `web\scripts\NetTalk_ARMiLogin.js`, copiado por el template al generar. Si se usan versiones comprimidas (`.gz`) de los scripts, volver a generarlas.
3. **No copiar ni sobrescribir** `NetTalkPasskeys.tps`. Es un dato de producción, igual que los demás `.tps`. Si ya existe se conserva; si no existe, el servidor lo crea en el primer alta. Incluirlo en el backup.
4. Comprobar que la carpeta donde corre el `.exe` permite escritura (allí se crea el `.tps`), o indicar otra ruta en el prompt **Credential file**.
5. Probar con un teléfono real: ingresar con contraseña, activar, cerrar sesión y entrar con el botón.

> **Nota:** Si el archivo de credenciales se pierde, nadie queda bloqueado: cada usuario entra con su contraseña y vuelve a activar la huella. El script detecta que el dispositivo tiene una clave que el servidor ya no conoce y vuelve a ofrecer el alta.

7.1 Servidor detrás de un proxy o reenvío

Si IIS (ARR), un balanceador o un túnel cambia el encabezado `Host` antes de llegar a NetTalk, el servidor no conocerá el dominio público. En ese caso, en la solapa **Domain** de la extension global completar:

Prompt	Ejemplo
RP id (domain)	`'portit.com.ar'` (el dominio público, sin `https://` ni puerto)
Expected origin	`'https://portit.com.ar'` (con `https://`; si usa un puerto distinto de 443, agregarlo)

> **Importante:** Las claves están ligadas al dominio. Las claves creadas en `localhost` o en un túnel de prueba **no sirven en producción**: cada usuario debe activar la huella de nuevo en el dominio real. Cambiar de dominio exige un alta nueva en cada dispositivo.

8. Aplicación de ejemplo

Junto con el template se entrega una aplicación mínima que ya tiene las tres extensions configuradas, para que el programador pueda ver el resultado antes de modificar su propia aplicación.

Dato	Valor
Nombre de la aplicación	NtLoginDemo.app
Carpeta donde la deja el instalador	`<Clarion>\ARMi\Nettalk_ARMiLogin`
Procedimiento de login	LoginForm
Página de destino	IndexPage
Usuario y contraseña de prueba	demo / demo

8.1 Cómo usarla

1. Abrir la aplicación de ejemplo en el IDE y generar / compilar (requiere `bcrypt.lib`).
2. Ejecutar el servidor y abrir `http://localhost:<puerto>` en el navegador.
3. Seguir el paso 9 del capítulo 4 para activar la huella y volver a ingresar con ella.
4. Revisar en el IDE cómo están configuradas las tres extensions y el embed, y repetirlo en la aplicación propia.

9. Solución de problemas

Síntoma	Causa probable y solución
No compila: `Unresolved External BCrypt...`	Falta `bcrypt.lib` en `<Clarion>\Accessory\Lib`. Una copia en la raíz de Clarion no la encuentra el enlazador.
Al generar aparece "bcrypt.lib not found" o "NetTalk_ARMiLogin.js not found"	Instalación incompleta: reinstalar NetTalk ARMi Login (ver 3.1).
Errores raros al compilar el `.clw` o `.inc` ("Illegal data type", "Field not found")	Los archivos Clarion deben tener fin de línea **CRLF**. Si se editaron con una herramienta que guarda solo LF, el compilador los lee como una única línea. Volver a guardarlos con CRLF.
El botón no aparece	La página no es un origen seguro (usar HTTPS o `http://localhost`; con `http://IP` o `http://nombre-de-pc` el navegador no permite WebAuthn). O el navegador cargó una copia vieja del `.js` (recargar con Ctrl+F5). O falta `NetTalk_ARMiLogin.js` en `web\scripts` (regenerar la aplicación para que el template lo copie).
El botón aparece pero al activar da error de dominio	Se está usando una IP como dirección. La clave necesita un nombre de dominio.
"Activate" no hace nada	Modo incógnito de Chrome (no permite crear claves de paso en el dispositivo): usar una pestaña normal. O el dispositivo no tiene bloqueo de pantalla / Windows Hello configurado.
Aparece "no longer registered on the server"	El servidor no tiene la credencial (archivo borrado o distinto). Ingresar con contraseña y activar de nuevo. Comprobar que el servidor lee siempre el mismo `.tps`.

Síntoma	Causa probable y solución
Con proxy / túnel falla el origen o el dominio	Completar **RP id** y **Expected origin** en la solapa Domain de la extension global (ver 7.1).
Se ingresa pero faltan datos de sesión	El embed `ARMi Login: fingerprint verified` está vacío o no carga las mismas variables que el login normal.
El ingreso se rechaza y no se ve por qué	Revisar el log de la aplicación: `fingerprint login refused: <motivo>`. Para ver el nombre del error del navegador, poner `debug: true` en la línea `var cfg = {...}` de `NetTalk_ARMiLogin.js` (solo mientras se prueba).

9.1 Códigos de error de la clase

El motivo del último rechazo está en `WAAuth.LastError` (código) y `WAAuth.LastErrorText` (texto).

Código	Constante	Significado
0	`WA:Ok`	Operación correcta.
1	`WA:Err:BadInput`	Datos incompletos o mal formados.
2	`WA:Err:ClientData`	Tipo, desafío u origen no coinciden.
3	`WA:Err:RpIdHash`	El dominio de la credencial no es el de esta aplicación.
4	`WA:Err:Flags`	Falta la presencia o la verificación del usuario.
5	`WA:Err:Unknown`	La credencial no está registrada en el servidor.
6	`WA:Err:Signature`	La firma no verifica.
7	`WA:Err:Counter`	El contador de firmas no creció (posible clon o repetición).
8	`WA:Err:Crypto`	Falló una llamada a `bcrypt.dll`.
9	`WA:Err:Store`	No se pudo abrir o escribir el archivo de credenciales.

10. Preguntas frecuentes

¿Qué pasa si el usuario cambia de teléfono?

Ingresa con contraseña en el teléfono nuevo y activa la huella allí. Un usuario puede tener varias huellas activadas (una por dispositivo); las anteriores siguen funcionando.

¿Qué pasa si cambia el dato que identifica al usuario?

Las huellas ya activadas conservan el valor que se guardó al activarlas (por ejemplo, el e-mail anterior) y el ingreso con huella va a devolver ese valor viejo. Hay que borrar las huellas de ese usuario (método DeleteCredentials) y pedirle que active la huella de nuevo. Por eso conviene guardar un dato que no cambie, como un ID interno.

¿Se puede quitar una huella?

La clase incluye el método `DeleteCredentials(usuario)`, que borra todas las huellas de un usuario. El template todavía no trae una pantalla para hacerlo; se puede llamar desde un procedimiento propio de administración.

¿Funciona con claves sincronizadas (iCloud, Google)?

Sí. Si el usuario tiene la clave sincronizada en otro dispositivo pero este navegador nunca la usó, el botón muestra el aviso "no activado" con la opción **Try anyway**, que permite probar con la clave sincronizada.

¿Es seguro guardar las credenciales en un .tps?

El archivo contiene solo claves públicas, contadores y datos informativos: no sirve para ingresar. Igual conviene protegerlo como los demás datos de la aplicación y mantener un backup.

¿Se puede usar sin la página "Offer"?

No: es la que ofrece y registra la huella. Sin ella ningún usuario podría activarla.

¿Hay otra forma de almacenar las credenciales que no sea un .tps?

Hoy no. Guardarlas en una tabla de la base de datos de la aplicación está previsto como mejora futura.

Apéndice A. Glosario

Término	Significado
WebAuthn	Estándar del W3C que permite a un sitio web verificar al usuario con un autenticador del dispositivo (huella, rostro, PIN).
Clave de paso (passkey)	Credencial de WebAuthn guardada en el dispositivo o sincronizada por la plataforma. Reemplaza a la contraseña.
RP id	Identificador del sitio ("relying party"): el nombre de dominio al que queda ligada la clave.
Origen (origin)	Esquema + dominio + puerto de la página (por ejemplo `https://portit.com.ar`).
Desafío (challenge)	Valor aleatorio de un solo uso que el servidor emite y el dispositivo firma, para evitar que se repita un ingreso.
ES256	Firma ECDSA con curva P-256 y SHA-256. Es el algoritmo que usa este template.
CNG	Cryptography API: Next Generation, la API criptográfica de Windows (`bcrypt.dll`).
Verificación de usuario (UV)	Confirmación local de la persona en el dispositivo: huella, rostro o PIN.